

EXPRESS ANALYTICS

AI/ML Engineer Intern

TAKE-HOME ASSIGNMENT

RAG-Based Technical Documentation Assistant

Build a Retrieval-Augmented Generation system with a self-corrective LangGraph workflow

Role: AI/ML Engineer - Intern

Duration: 2 days

Submission: GitHub Repository + README

Stack: Python, LangGraph, FastAPI

Please read the entire document before starting.

Overview

Build a RAG-powered technical documentation assistant that can answer questions about a set of technical documents (e.g., Python library docs, API references, or framework guides). The system should use a LangGraph workflow with retrieval, document grading, and answer generation nodes. It should be served via a simple FastAPI application.

This assignment evaluates your ability to build a RAG pipeline, work with LangGraph's graph-based workflows. There is no single correct architecture — we want to see how you think.

Scenario

You are provided with a small corpus of technical documentation (3–5 documents, such as pages from a Python library's official docs, README files, or API references). A user submits a natural language question, and your system should retrieve the most relevant chunks, verify their relevance, generate an answer with citations, and handle cases where the retrieved context is insufficient.

You may choose the corpus yourself — for example, you could use documentation from LangChain, FastAPI, Pydantic, or any other technical project you find interesting. Provide the documents (or URLs to fetch them) in your repository.

System Architecture

LangGraph Workflow

Your system should be implemented as a LangGraph StateGraph with the following nodes. You may add additional nodes if you see fit.

Node 1: Query Analysis

Takes the user's raw question and prepares it for retrieval. This node should:

- Rewrite or expand the query to improve retrieval quality (e.g., add synonyms, clarify ambiguity)

- Optionally classify the query type (conceptual, how-to, troubleshooting, API reference) to guide downstream processing

Node 2: Retrieval

Searches the vector store for relevant document chunks. This node should:

- Use a vector store (e.g., ChromaDB, FAISS) to perform similarity search
- Return the top-k most relevant chunks along with their source metadata

Node 3: Document Grading

Evaluates whether the retrieved documents are actually relevant to the question. This is the self-corrective component of the pipeline. The node should:

- Use an LLM to grade each retrieved chunk as “relevant” or “irrelevant”
- If all documents are irrelevant, route to a fallback path (e.g., rewrite the query and re-retrieve, or return an “I don’t know” response)
- If some documents are relevant, filter out the irrelevant ones and proceed

Node 4: Generation

Generates the final answer using the relevant documents as context. This node should:

- Use an LLM to generate a clear, accurate answer grounded in the retrieved context
- Include citations or references to the source documents in the response

Conditional Edges (Routing Logic)

Your graph should include at least one conditional edge that routes based on the document grading outcome:

- If relevant documents are found → proceed to Generation
- If no relevant documents are found → rewrite query and re-retrieve (with a retry limit).

Think carefully about your state schema. What data flows between nodes? How do you track the number of retries? This is a core evaluation criterion.

Document Ingestion Pipeline

Before the workflow can run, you need to ingest and index your technical documents. Your ingestion pipeline should:

1. Load documents from files (Markdown, text, or HTML) or fetch them from URLs
2. Split documents into chunks using an appropriate strategy (consider chunk size, overlap, and how technical content should be split)
3. Generate embeddings for each chunk using an embedding model (e.g., OpenAI embeddings, sentence-transformers, or any other provider)
4. Store the embeddings in a vector store (ChromaDB or FAISS recommended for simplicity)

The ingestion can be a standalone script or run as part of application startup. Document your chunking strategy and explain why you chose it.

API Requirements

Serve the system via FastAPI. At minimum, your API should support the following endpoints:

Method	Endpoint	Purpose	Notes
POST	/query	Submit a question	Returns answer with sources
POST	/ingest	Ingest new documents	Accepts file uploads or URLs
GET	/documents	List indexed documents	Shows what's in the corpus
POST	/feedback	Submit feedback on an answer	Thumbs up/down + optional comment

You may add additional endpoints as you see fit. Consider error handling, input validation, and meaningful HTTP status codes.

Bonus (Optional)

These are not required but will strengthen your submission if you have time:

- Hallucination check: Add a node that verifies the generated answer is actually supported by the retrieved context (inspired by Self-RAG)
- Web search fallback: If the vector store has no relevant results, fall back to a web search (e.g., using Tavily, Serper or Exa) before generating the answer
- Conversation memory: Support follow-up questions by maintaining chat history within a session
- Simple UI: A minimal Streamlit or Gradio frontend for interactive Q&A

Deliverables

- A GitHub repository (public or private — invite us if private) containing all source code
- A README.md with: project overview, architecture diagram or description, setup instructions (environment, API keys, dependencies), how to run the application, example API requests and responses, and a section on design decisions and tradeoffs
- A working FastAPI application that can be run locally
- The document corpus you used (or a script to fetch it)
- A brief write-up (can be in the README) explaining: your thought process and reasoning behind the architecture and the components of your workflow, what you would improve with more time, any assumptions you made, and your chunking/embedding strategy choices

LangGraph Cookbooks & References

The following official and community resources will help you understand the patterns needed for this assignment. We strongly recommend reviewing these before you start coding.

Official LangGraph Tutorials

- **Agentic RAG Tutorial** — Build a custom RAG agent with LangGraph (retrieval, grading, query rewriting)
<https://docs.langchain.com/oss/python/langgraph/agent-rag>
- **Adaptive RAG Tutorial** — Combines query routing with self-corrective RAG (closest to this assignment's architecture)
https://langchain-ai.github.io/langgraph/tutorials/rag/langgraph_adaptive_rag/
- **Corrective RAG (CRAG) Notebook** — Document grading and query rewriting patterns from the CRAG paper
https://github.com/langchain-ai/langgraph/blob/main/examples/rag/langgraph_crags.ipynb
- **Self-RAG Blog Post** — Overview of self-reflective RAG concepts and LangGraph implementation
<https://blog.langchain.com/agent-rag-with-langgraph/>

Community Tutorials

- **LangChain Open Tutorial — Adaptive RAG** — Step-by-step walkthrough with groundedness checking and web search modules
<https://langchain-opentutorial.gitbook.io/langchain-opentutorial/17-langgraph/02-structures/06-langgraph-agent-rag>
- **FutureSmart AI — LangGraph RAG Agent Tutorial** — Covers basics to multi-agent RAG, good for understanding the progression
<https://blog.futuremart.ai/langgraph-rag-agent-tutorial-basics-to-advanced-multi-agent-ai-chatbot>

Reference: Tools & APIs You Can Use

Below are some tools and APIs that may be useful. You are not limited to these — use whatever you think is appropriate.

LLM Providers (pick one)

Provider	Notes	Free Tier
OpenAI	GPT-4o-mini recommended for cost efficiency	Pay-as-you-go (low cost for this project)
Anthropic	Claude 3.5 Sonnet / Haiku	Pay-as-you-go
Groq	Very fast inference, Llama & Mixtral models	Generous free tier
Google Gemini	Gemini Flash for fast, cheap inference	Free tier available

Vector Stores

Tool	Notes	Free Tier
ChromaDB	Simple, runs locally, great for prototyping	Free, open source
FAISS	Facebook's similarity search, very fast	Free, open source

Web Search (for bonus fallback)

API	Purpose	Free Tier
Tavily (tavily.com)	AI-optimized web search	1,000 searches/month free
Serper (serper.dev)	Google Search results via API	2,500 free queries on signup

Important Notes

- You may use any LLM provider for the pipeline (OpenAI, Anthropic, Groq, Google, etc.)
- Focus on getting the core RAG pipeline right before attempting bonus features
- If you run into time constraints, prioritize the LangGraph workflow + document grading logic over additional features
- We value clear thinking and honest documentation of tradeoffs over feature completeness
- Don't hesitate to ask clarifying questions before starting

Good luck! We look forward to reviewing your work.